SPEC: Web ôn thi kiểu Duolingo (Spring Boot + React)

Đặc tả đầy đủ để giao cho AI lập trình (Claude Code, Cursor...). Lưu thành `SPEC.md` trong thư mục dự án và giao từng bước theo mục "Thứ tự thực hiện".

1. Vai trò và mục tiêu

Bạn là kỹ sư full-stack giàu kinh nghiệm. Hãy xây dựng một web app ôn thi kiểu Duolingo, **dùng cá nhân (một người dùng duy nhất)**, truy cập được cả trên máy tính và điện thoại.

Người dùng tải tài liệu ôn thi lên (PDF, Word, .txt/.md hoặc dán text). Hệ thống dùng AI tự chia thành lộ trình: **Unit (chương) → Section (nhóm chủ đề) → Lesson (bài nhỏ, học khoảng 3 phút, 5–8 Item)**. Mỗi ngày hệ thống tự dựng phiên học gồm bài mới và ôn lại theo thuật toán nhớ ngắt quãng (SRS), dựa trên ngày thi và số phút học mỗi ngày. Giao diện bằng tiếng Việt, ưu tiên mobile-first.

Quy tắc làm việc: làm tuần tự theo mục 14, mỗi bước phải chạy được, có test cho logic quan trọng, không viết code giả hoặc để TODO bỏ ngỏ. Khi gặp điểm chưa rõ, chọn phương án hợp lý nhất, ghi giả định vào phần tóm tắt và không dừng lại hỏi.

2. Công nghệ

- **Backend:** Java 21, Spring Boot 3 (Maven), Spring Web, Spring Data JPA, Spring Security, Bean Validation, Flyway.
- **Database:** PostgreSQL (dev chạy bằng Docker Compose).
- **Trích text:** Apache PDFBox (PDF), Apache POI (.docx), đọc trực tiếp .txt/.md.
- **AI:** Anthropic Messages API, model `claude-sonnet-4-6`, gọi bằng RestClient. Khóa đọc từ biến môi trường `ANTHROPIC_API_KEY`. Ép trả JSON theo schema, parse và validate bằng Jackson + Bean Validation; lỗi thì thử lại tối đa 2 lần. Giới hạn 2 lệnh AI chạy song song.
- **Frontend:** React + TypeScript + Vite + Tailwind CSS, react-router, TanStack Query. Làm **PWA** (manifest + service worker) để thêm vào màn hình chính điện thoại.
- **Cấu trúc repo:** monorepo gồm `/backend` và `/frontend`. Khi build, frontend được copy vào thư mục static của Spring để deploy thành MỘT container. Cung cấp `docker-compose.yml` cho dev (Postgres) và README hướng dẫn cài, cấu hình, chạy.

- **Dữ liệu năm hoàn toàn trên server.** Không lưu dữ liệu học ở trình duyệt (chỉ cache tạm bằng TanStack Query) để máy tính và điện thoại dùng chung một bộ dữ liệu.

3. Bảo mật (một người dùng)

- Không có đăng ký. Một mật khẩu duy nhất đặt qua biến môi trường `APP_PASSWORD`.
- `POST /api/login` kiểm tra mật khẩu, trả cookie phiên HttpOnly + Secure + SameSite=Lax. Mọi `/api/**` khác bắt buộc có phiên hợp lệ.
- Giới hạn thử sai mật khẩu: 5 lần/phút. CORS chỉ cho phép đúng domain frontend.
- Không bao giờ để lộ API key hay mật khẩu trong code, log hoặc frontend.
- Giới hạn upload 20MB, chỉ nhận pdf/docx/txt/md.

4. Mô hình dữ liệu (JPA entity + migration Flyway)

Dùng UUID làm khóa chính. Lưu thời gian dạng UTC; tính "ngày" (streak, lịch) theo múi giờ cấu hình được, mặc định `Asia/Ho_Chi_Minh`.

- **Course:** id, title, examDate, dailyMinutes, createdAt, rawText, status.
- **Unit:** id, courseId, order, title.
- **Section:** id, unitId, order, title.
- **Lesson:** id, sectionId, order, title, sourceText, status (locked | ready | generating | done), scheduledDate, bestScore, completedAt.
- **Item:** id, lessonId, prompt, intro, answer, distractors (3), explanation, sourceQuote.
- **ItemState:** itemId, ease (mặc định 2.5), intervalDays, repetitions, lapses, dueAt, lastResult, introduced (boolean).
- **ReviewLog:** id, itemId, correct, responseMs, at.
- **Profile:** xp, streak, bestStreak, lastActiveDate, dailyGoalXp (mặc định 20), xpToday, timezone.

Item là đơn vị kiến thức nhỏ nhất. SRS theo dõi theo Item, không theo bài học hay câu hỏi.

5. Luồng 1: Tạo khóa học từ tài liệu

1. Màn "Khóa học mới": tải file hoặc dán text, nhập tên khóa, ngày thi (date picker), số phút học mỗi ngày (10/15/20/30/45/60).

2. Server trích text và báo số ký tự đọc được. Nếu PDF là ảnh quét không có text thì báo lỗi rõ ràng.
3. Gọi AI dựng **mục lục**. Nếu tài liệu dài, chia thành các đoạn theo heading/độ dài (~6000 ký tự, chồng lấn nhỏ), gửi từng đoạn rồi gộp và loại trùng. Kết quả JSON: units[] → sections[] → lessons[]; mỗi lesson có `title` và `sourceText` (phần tài liệu liên quan), chứa lượng kiến thức đủ cho 5–8 Item.
4. Hiển thị "Đường học" để người dùng xem và chỉnh: đổi tên, xóa, gộp hoặc tách bài. Bấm "Bắt đầu" để chốt.
5. Hệ thống xếp lịch (mục 6). Item của từng Lesson **không sinh ngay** mà sinh theo kiểu lazy (trước một ngày hoặc khi người dùng bấm vào bài).
6. Tạo khóa học và sinh Item là tác vụ lâu: chạy bất đồng bộ (`@Async`), trả về taskId, frontend poll trạng thái để hiện tiến độ. Trước khi xử lý tài liệu rất dài, hiển thị ước tính thời gian/chi phí và cho phép hủy.

6. Luồng 2: Xếp lịch

- Số ngày học = từ hôm nay đến ngày thi, trừ N ngày cuối chỉ ôn (N = 3 nếu còn từ 14 ngày, N = 2 nếu ít hơn, N = 0 nếu dưới 5 ngày).
- Số Lesson mới mỗi ngày = ceil(tổng Lesson chưa học / số ngày học). Gán `scheduledDate` theo đúng thứ tự.
- Nếu bỏ lỡ ngày, tự dồn lịch vào các ngày còn lại, không để nợ bài chồng chất.
- Ước lượng tải: khoảng 3 phút cho mỗi Lesson mới và 0,3 phút cho mỗi Item ôn. Nếu vượt `dailyMinutes`, cảnh báo lịch quá tải và gợi ý tăng thời gian học hoặc dời ngày thi.
- Màn Lịch hiển thị từng ngày: số bài mới và số Item dự kiến phải ôn.

7. Luồng 3: Sinh Item cho một Lesson

Gọi AI với `sourceText` của Lesson, trả về 5–8 Item theo schema:

- `prompt` : câu hỏi rõ ràng, độc lập, không phụ thuộc câu khác.
- `intro` : 1–3 câu giải thích khái niệm cho lần đầu gặp.
- `answer` : đáp án đúng.
- `distractors` : 3 đáp án nhiễu hợp lý, cùng loại với đáp án đúng, không quá lộ.
- `explanation` : vì sao đúng (ngắn).
- `sourceQuote` : trích nguyên văn tối đa 25 từ từ tài liệu làm căn cứ.

Ràng buộc trong prompt gửi AI: chỉ dùng thông tin có trong tài liệu, không bịa; mỗi Item chỉ kiểm tra một ý; đáp án đúng không luôn là đáp án dài nhất; viết tiếng Việt, giữ nguyên thuật ngữ gốc nếu tài liệu dùng. **Nếu `sourceQuote` không xuất hiện trong `sourceText` thì loại Item đó.**

8. Dạng bài (chỉ có 2, không thêm)

1. **Thẻ giới thiệu:** hiện cho Item chưa từng học (`introduced = false`): prompt + intro + sourceQuote, nút "Đã hiểu, kiểm tra thử".
2. **Trắc nghiệm 4 đáp án** (xáo ngẫu nhiên vị trí): dùng cho MỌI Item, kể cả khi ôn lại.

Không có điền chỗ trống, tự gõ hay sắp xếp. Độ khó được điều chỉnh bằng SRS (tần suất gặp lại), không bằng đổi dạng bài.

9. Luồng 4: Phiên học hằng ngày

Nút lớn "Học hôm nay" ở trang chủ. Backend dựng hàng đợi theo ưu tiên, cắt theo `dailyMinutes`:

1. Item đến hạn (`dueAt` đã tới), ưu tiên quá hạn lâu nhất.
2. Lesson mới có `scheduledDate` đến hôm nay (gồm cả bài bị bỏ lỡ).
3. Nếu còn thời gian: Item yếu nhất (lapses cao, ease thấp).

Nếu hôm nay phải ôn quá nhiều, hoãn bớt Lesson mới thay vì dồn thêm.

Vòng lặp một phiên: (Thẻ giới thiệu nếu cần) → câu trắc nghiệm → chọn xong hiện ngay đúng/sai + explanation + sourceQuote → gửi kết quả lên server cập nhật SRS → nếu sai thì đẩy Item xuống cuối hàng để gặp lại trong phiên (tối đa 2 lần mỗi Item/phiên) → nút "Tiếp".

Có thanh tiến độ phiên và nút thoát (xác nhận, lưu tiến độ). Cuối phiên: màn kết quả gồm số đúng/sai, XP nhận, streak, danh sách Item sai kèm giải thích.

Ngoài ra có **"Ôn tập tự do"**: chọn một Unit/Section để luyện thêm, có công tắc bật/tắt việc tính vào SRS.

10. Thuật toán SRS (SM-2 rút gọn, viết ở backend, có JUnit test)

Quy đổi `quality`: sai = 0; đúng nhưng trả lời quá 15 giây = 3; đúng = 4; đúng và dưới 5 giây = 5. Item gặp lần đầu trong phiên mà sai thì chưa tính là lapse.

- Nếu quality < 3: `repetitions = 0`, `intervalDays = 1`, `lapses++`.

- Ngược lại: `repetitions++`; lần 1 → 1 ngày, lần 2 → 3 ngày, các lần sau → `round(intervalDays * ease)`.
- `ease = max(1.3, ease + 0.1 - (5 - q) * (0.08 + (5 - q) * 0.02))`.
- `dueAt = now + intervalDays`; nếu vượt ngày thi thì kéo về trước ngày thi.
- Ghi mọi lượt trả lời vào ReviewLog.

11. Bài kiểm tra đầu vào (tùy chọn)

Sau khi tạo khóa, cho phép làm thử 5 câu đại diện mỗi Unit (AI sinh nhanh). Unit nào đúng từ 80% trở lên thì đánh dấu "đã quen": giảm số Lesson mới của Unit đó và đưa các Item vào SRS với `intervalDays = 3` thay vì bắt học từ đầu.

12. Gamification

- **XP:** +10 mỗi câu đúng lần đầu, +5 nếu đúng sau khi lặp lại, +20 thưởng khi hoàn thành bài không sai.
- **Streak:** tăng khi hoàn thành ít nhất một phiên trong ngày (theo múi giờ địa phương), mất nếu bỏ qua một ngày. Hiện ngọn lửa và số ngày.
- **Mục tiêu ngày** (mặc định 20 XP): vòng tròn tiến độ ở trang chủ.
- **Không có tim/hearts**, không chặn người dùng học.
- Hiệu ứng nhẹ khi đúng/sai (đổi màu, rung nhẹ); âm thanh tùy chọn bật/tắt trong Cài đặt.

13. API, màn hình và giao diện

API (REST, JSON):

- `POST /api/login`
- `POST /api/courses` (multipart), `GET /api/courses`, `GET /api/courses/{id}/path`, `PATCH` unit/section/lesson, `POST /api/courses/{id}/start`, `GET /api/tasks/{id}`
- `GET /api/schedule`
- `GET /api/session/today` (trả hàng đợi), `POST /api/session/answer` (cập nhật SRS, XP), `POST /api/session/finish`
- `GET /api/stats`, `GET/PUT /api/settings`, `GET /api/export`, `POST /api/import`

Màn hình:

1. **Trang chủ:** tên khóa, đếm ngược ngày thi, nút "Học hôm nay" (kèm số Item cần ôn và số bài mới), streak, vòng XP ngày.
2. **Đường học:** Unit → Section → Lesson dạng đường nối dọc kiểu Duolingo; trạng thái khóa / sẵn sàng / đã xong (kèm sao theo điểm). Bấm Lesson sẵn sàng để học riêng.
3. **Phiên học và Kết quả.**
4. **Lịch:** kế hoạch từng ngày đến ngày thi.
5. **Thống kê:** số Item đã nhớ vững (`intervalDays >= 21`), biểu đồ số Item ôn theo ngày, độ chính xác theo Unit, top 10 Item hay sai.
6. **Khóa học:** tạo mới, chuyển đổi, xóa nhiều khóa.
7. **Cài đặt:** số phút/ngày, ngày thi, mục tiêu XP, âm thanh, giao diện sáng/tối/tự động, múi giờ, xuất/nhập JSON, xóa toàn bộ dữ liệu.

Giao diện: thân thiện, nhiều khoảng trắng, nút bo tròn lớn dễ bấm bằng ngón tay, thanh điều hướng dưới cùng trên mobile (Trang chủ, Đường học, Ôn tập, Thống kê). Dùng biến màu CSS hỗ trợ sáng/tối. Xanh lá cho đúng, đỏ nhạt cho sai, một màu nhấn chính. Phông chữ hệ thống hỗ trợ tiếng Việt tốt. Có skeleton/loading rõ ràng khi chờ AI và nút thử lại khi AI lỗi.

Xử lý lỗi: mất mạng hoặc AI lỗi thì không mất tiến độ đã học; Lesson đã có Item vẫn học được. Validate toàn bộ đầu vào ở backend.

14. Thứ tự thực hiện

Làm xong và chạy được mỗi bước rồi mới sang bước tiếp. Cuối mỗi bước tóm tắt ngắn: đã làm gì, giả định gì, chưa làm gì.

1. Khởi tạo monorepo: Spring Boot + Postgres (docker-compose) + Flyway schema; React + Vite + Tailwind; đăng nhập bằng mật khẩu.
2. Entity + SRS service + JUnit test. Seed khoảng 20 Item mẫu.
3. API phiên học (today / answer / finish) và màn hình phiên học: thẻ giới thiệu, trắc nghiệm, hàng đợi lặp lại, màn kết quả.
4. XP, streak, mục tiêu ngày, trang chủ, thống kê cơ bản.
5. Đường học (Unit/Section/Lesson), lịch, thuật toán xếp lịch và dồn lịch.
6. Trích text PDF/docx + client Claude + validate JSON + retry + giới hạn song song.
7. Pipeline tạo khóa học bằng AI (mục lục, chỉnh sửa, sinh Item lazy), bài kiểm tra đầu vào.
8. Cài đặt, xuất/nhập JSON, PWA, tinh chỉnh mobile, xử lý lỗi, README.

9. *(Để sau)* Dockerfile production nhiều giai đoạn và hướng dẫn deploy (Render/Railway/Fly.io + Postgres), biến môi trường: `ANTHROPIC_API_KEY`, `APP_PASSWORD`, chuỗi kết nối DB.

15. Tiêu chí nghiệm thu

- Tải một tài liệu thật vài chục trang, ra lộ trình hợp lý trong dưới 2 phút.
- Học xong một Lesson mới: Item xuất hiện lại đúng hạn (1 ngày, rồi 3 ngày…); chỉnh ngày hệ thống để kiểm chứng.
- Sai một Item thì gặp lại ngay trong phiên và lần ôn sau đến sớm hơn.
- Streak, XP, mục tiêu ngày cập nhật đúng; qua nửa đêm giờ địa phương thì sang ngày mới.
- Đăng nhập trên điện thoại và máy tính thấy cùng một tiến độ. Xuất rồi nhập JSON khôi phục được toàn bộ.
- Dùng mượt trên màn hình 375px và cài được như một ứng dụng (PWA).
- Logic SRS, xếp lịch, XP/streak có unit test và test đều pass.